

Java Classes

(Refer Java-the-complete-reference-eleventh-edition, under “Introducing Classes” page 210.)

- Class is the core of Java.
- Any concept or a solution you wish to implement in a Java program must be encapsulated within a class.
- Class acts as a blue-print or template for constructing the objects.
 - Example, Before constructing a house or building, we will have a blue-print document.
- The most important thing to understand about a class is that it defines a new data type.
- Once defined, the new type (class) can be used to create objects of that type.
- Thus class is a template for an object and object is an instance of a class.
- The class can be declared using **class key word**. A simplified general form of a class definition is shown here.
- The data, or variables, defined within a class are called **instance variables**.
- Collectively, the methods and variables defined within a class are called **members of the class**.

Classes can (and usually do) get much more complex. A simplified general form of a class definition is shown here:

```
class classname {  
    type instance-variable1;  
    type instance-variable2;  
    // ...  
    type instance-variableN;  
  
    type methodname1(parameter-list) {  
        // body of method  
    }  
    type methodname2(parameter-list) {  
        // body of method  
    }  
    // ...  
    type methodnameN(parameter-list) {  
        // body of method  
    }  
}
```

Java Classes

- In most classes, the instance variables are acted upon and accessed by the methods defined for that class. Thus, as a general rule, it is the methods that determine how a class's data can be used.
- Variables defined within a class are called **instance variables** because each instance of the class (that is, each object of the class) contains its **own copy of these variables**. Thus, the data for one object is separate and unique from the data for another.
- Thus a class creates a new datatype that can be used to create objects. That is, a class creates a logical framework that defines the relationship between its members.

```
Rectangle.java > ...
1 public class Rectangle {
2
3     double length;
4     double breadth;
5
6     void calculateArea() {
7         double area = length * breadth;
8         System.out.println("The area of Rectangle is: " + area);
9     }
10 }
```

Class Name

Data attributes (or Variables)

Member Functions (or Methods)

```
Demo.java > ...
1 public class Demo {
2     Run | Debug
3     public static void main(String args[]) {
4         Rectangle myrec = new Rectangle(); //first object created
5         myrec.length = 20;
6         myrec.breadth = 30;
7         myrec.calculateArea();
8     }
9 }
10
```

Object of class Rectangle

Using object to access variables of methods of class Rectangle

Java Classes -Simple Box Class

- Box that defines three instance variables: width, height, and depth. Currently, Box does not contain any methods

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

- As stated, class defines a new data-type. In this case new data-type is Box. declaration only creates a template, It does not create an actual object.
- To actually create a Box object, we use the following statement.

```
Box mybox = new Box(); // create a Box object called mybox
```

- After this statement executes, mybox will refer to an instance of Box.

Class

Java Classes -Simple Box Class

Here is a complete program that uses the **Box** class:

```
/* A program that uses the Box class.
```

```
    Call this file BoxDemo.java
*/
class Box {
    double width;
    double height;
    double depth;
}

// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;

        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + vol);
    }
}
```



BoxDemo.java

Java Classes -Simple Box Class

In the preceding sample programs, a line similar to the following is used to declare an object of type **Box**:

```
Box mybox = new Box();
```

This statement combines the two steps just described. It can be rewritten like this to show each step more clearly:

```
Box mybox; // declare reference to object  
mybox = new Box(); // allocate a Box object
```

- The first line declares **mybox** as an reference to an object of type **Box**. At this point, **mybox** does not yet refer to an actual object.
- The next line allocates memory for an object and assigns reference to it to **mybox**.
- After the second line executes, we can use **mybox** as if it is a actual object.
- But in reality, **mybox** simply holds the memory address of the actual **Box** object.
- This is similar to declaring a pointer to a struct in C and allocating memory using malloc.

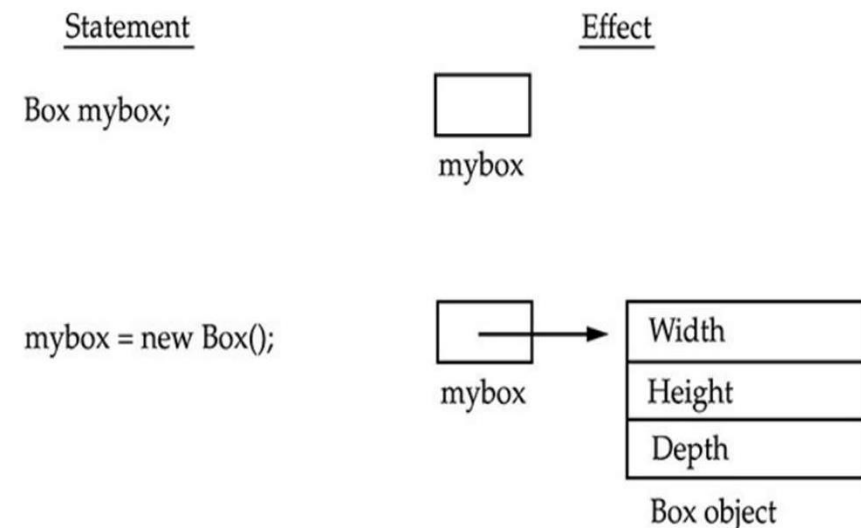


Figure 6-1 Declaring an object of type **Box**

Java Classes -Simple Box Class - Multiple Box Objects

- This program declares two box objects.

```
class BoxDemo2 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // compute volume of first box
        vol = mybox1.width * mybox1.height * mybox1.depth;
        System.out.println("Volume is " + vol);

        // compute volume of second box
        vol = mybox2.width * mybox2.height * mybox2.depth;
        System.out.println("Volume is " + vol);
    }
}
```

Java Classes -Simple Box Class - Box Constructor

A Closer Look at new

As just explained, the **new** operator dynamically allocates memory for an object. In the context of an assignment, it has this general form:

class-var = new classname ();

- Here, class-var is a variable of the class type being created.
- The classname is the name of the class that is being instantiated.
- The classname followed by the parentheses specifies the constructor for the class.
- The constructor is a method either implicitly defined by the compiler by default or explicitly defined by the code.
- The constructor defines **what occurs when an object of a class is created**.
- e.g. **BOX mybox = new Box()**
- Most real-world classes explicitly define their own constructors within their class definition.
- If no explicit constructor is defined or specified, then Java compiler will automatically supply a **default constructor**. This is the case with Box class.

Java Methods

- The class consist of two things.
 - Instance variables (class Member Variables)
 - Methods (class Member Functions)
- Most of the time we use methods to access the instance variables defined by the class.
- In fact methods define the interface to most classes. This allows class implementor to hide the specific layout of the internal data structures to create cleaner abstractions.
- In addition to defining methods that provide access to data, we can also define private methods that are used internally by the class itself.

Java Methods Adding Methods to Box class

- Adding Methods to the Box class.
- The previous Box example is reproduced below again.
- Note that the computation of volume of the Box is happening inside the BoxDemo class instead of Box class.

```
class Box {
    double width;
    double height;
    double depth;

    // display volume of a box
    void volume() {
        System.out.print("Volume is ");
        System.out.println(width * height * depth);
    }
}
```

Here is a complete program that uses the **Box** class:

```
/* A program that uses the Box class.

    Call this file BoxDemo.java
*/
class Box {
    double width;
    double height;
    double depth;
}

// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;

        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + vol);
    }
}
```

Java Methods Adding Methods to Box class

- Adding Methods to the Box class.
- The previous Box example is reproduced below again.
- Note that the computation of volume of the Box is happening inside the BoxDemo class instead of Box class.
- Updated Box class with method is shown below.

```
// This program includes a method inside the box class.

class Box {
    double width;
    double height;
    double depth;

    // display volume of a box
    void volume() {
        System.out.print("Volume is ");
        System.out.println(width * height * depth);
    }
}

class BoxDemo3 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // display volume of first box
        mybox1.volume();

        // display volume of second box
        mybox2.volume();
    }
}
```

Java Methods

Adding Methods to Box class

- Returning a value from the method.

-

```
// Now, volume() returns the volume of a box.

class Box {
    double width;
    double height;
    double depth;

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}

class BoxDemo4 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

Java Methods Adding Methods to Box class

- Adding a method that takes parameters
- As you can see the **setDim** method is used to set the dimensions of each Box.

```
// This program uses a parameterized method.

class Box {
    double width;
    double height;
    double depth;

    // compute and return volume
    double volume() {
        return width * height * depth;
    }

    // sets dimensions of box
    void setDim(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
}

class BoxDemo5 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // initialize each box
        mybox1.setDim(10, 20, 15);
        mybox2.setDim(3, 6, 9);

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

Java Methods - Constructors

- Constructor is a special method that is used to automatically initialize the objects.
- Constructor is called automatically when the object is created using new operator.
- Constructor has a name which is same as class name.
 - `Box mybox = new Box();`
- When the constructor is not provided, the java compiler provides the **default constructor** which initializes the class members to their default values.
 - In the Box class, we have not provided any explicit constructor. So default constructor provided by the compiler is used when the object is created.

Java Methods - Parametrized Constructors

Java Methods - **this** Keyword

- The **this** keyword and is a reference variable that refers to the current object in a method or a constructor.
- The most common use of **this** keyword is to eliminate the confusion between the class attributes and the parameters with the same name.
- **this** can also be used to
 - Invoke the current class constructor.
 - Invoke the current class method.
 - Return the current class object
 - Pass an argument in a method call
 - Pass an argument in a constructor call

Usage of Java **this** Keyword

There can be a lot of usage of java **this** keyword. In java, **this** is a reference variable that refers to the current object.

01	this can be used to refer current class instance variable.	04	this can be passed as an argument in the method call.
02	this can be used to invoke current class method (implicitly)	05	this can be passed as argument in the constructor call.
03	this() can be used to invoke current class Constructor.	06	this can be used to return the current class instance from the method

<https://www.geeksforgeeks.org/this-reference-in-java/>

<https://www.javatpoint.com/this-keyword>

Java Methods - this Keyword - Examples

- 1. Using 'this' keyword to refer to current class instance variables

```
// Java code for using 'this' keyword to
// refer current class instance variables
class Test {
    int a;
    int b;

    // Parameterized constructor
    Test(int a, int b)
    {
        this.a = a;
        this.b = b;
    }

    void display()
    {
        // Displaying value of variables a and b
        System.out.println("a = " + a + " b = " + b);
    }

    public static void main(String[] args)
    {
        Test object = new Test(10, 20);
        object.display();
    }
}
```

Java Methods - this Keyword - Examples

- 2. Using this() to invoke current class constructor

```
// Java code for using this() to
// invoke current class constructor
class Test {
    int a;
    int b;

    // Default constructor
    Test()
    {
        this(10, 20);
        System.out.println(
            "Inside default constructor \n");
    }

    // Parameterized constructor
    Test(int a, int b)
    {
        this.a = a;
        this.b = b;
        System.out.println(
            "Inside parameterized constructor");
    }

    public static void main(String[] args)
    {
        Test object = new Test();
    }
}
```

Java Methods - this Keyword - Examples

- 3. Using 'this' keyword to return the current class instance

```
// Java code for using 'this' keyword
// to return the current class instance
class Test {
    int a;
    int b;

    // Default constructor
    Test()
    {
        a = 10;
        b = 20;
    }

    // Method that returns current class instance
    Test get() { return this; }

    // Displaying value of variables a and b
    void display()
    {
        System.out.println("a = " + a + " b = " + b);
    }

    public static void main(String[] args)
    {
        Test object = new Test();
        object.get().display();
    }
}
```

Java Methods - this Keyword - Examples

- 4. Using 'this' keyword as a method parameter

```
// Java code for using 'this'
// keyword as method parameter
class Test {
    int a;
    int b;

    // Default constructor
    Test()
    {
        a = 10;
        b = 20;
    }

    // Method that receives 'this' keyword as parameter
    void display(Test obj)
    {
        System.out.println("a = " + obj.a
                           + " b = " + obj.b);
    }

    // Method that returns current class instance
    void get() { display(this); }

    // main function
    public static void main(String[] args)
    {
        Test object = new Test();
        object.get();
    }
}
```

Java Methods - this Keyword - Examples

- 5. Using 'this' keyword to invoke the current class method

```
// Java code for using this to invoke current
// class method
class Test {

    void display()
    {
        // calling function show()
        this.show();

        System.out.println("Inside display function");
    }

    void show()
    {
        System.out.println("Inside show function");
    }

    public static void main(String args[])
    {
        Test t1 = new Test();
        t1.display();
    }
}
```

Java Methods - this Keyword - Examples

- 6. Using 'this' keyword as an argument in the constructor call

```
// Java code for using this as an argument in constructor
// call
// Class with object of Class B as its data member
class A {
    B obj;

    // Parameterized constructor with object of B
    // as a parameter
    A(B obj)
    {
        this.obj = obj;

        // calling display method of class B
        obj.display();
    }
}
```

```
class B {
    int x = 5;

    // Default Constructor that create an object of A
    // with passing this as an argument in the
    // constructor
    B() { A obj = new A(this); }

    // method to show value of x
    void display()
    {
        System.out.println("Value of x in Class B : " + x);
    }

    public static void main(String[] args)
    {
        B obj = new B();
    }
}
```

Java Methods - this Keyword - Pros & Cons

Advantages of using 'this' reference

There are certain advantages of using 'this' reference in Java as mentioned below:

1. It helps to distinguish between instance variables and local variables with the same name.
2. It can be used to pass the current object as an argument to another method.
3. It can be used to return the current object from a method.
4. It can be used to invoke a constructor from another overloaded constructor in the same class.

Disadvantages of using 'this' reference

Although 'this' reference comes with many advantages there are certain disadvantages of also:

1. Overuse of this can make the code harder to read and understand.
2. Using this unnecessarily can add unnecessary overhead to the program.
3. Using this in a static context results in a compile-time error.
4. Overall, this keyword is a useful tool for working with objects in Java, but it should be used judiciously and only when necessary.

Java Garbage Collection

<https://www.geeksforgeeks.org/garbage-collection-java/>